Segurança e Privacidade (TP1)

Integrantes do - Grupo PL3G1:
Ezequiel Tchimbaya Cachapeu Paulo - (202400891),
Pedro da Graça - (202406955), Victor de Vargas - (202400863)

March 2026

INTRODUÇÃO

Este trabalho tem como objetivo apresentar os resultados obtidos pela equipa supracitada quanto ao Trabalho Prático nº 01 da unidade curricular de Segurança e Privacidade. Além dos resultados, também é detalhado o processo de elaboração de cada uma das resoluções que a equipa teve que passar para alcançar tais resultados em questão de eficiência computacional e matemática, bem como a inclusão da descrição da configuração experimental (como, características do computador, sistema operacional, versões de software e etc.).

Configuração do ambiente

Por questões de segurança pensamos em usar uma Máquina Virtual, no entanto, para testes de desempenho (benchmarking), usar uma Máquina Virtual (VM) exige cautela porque a virtualização altera a forma como o hardware é medido. Por está razão não usamos uma, mas criamos um ambiente virtual Conda para o nosso projecto:

Python = 3.12
Neste ambiente instalamos as seguintes biblíotecas: **cryptography (hazmat layer), timeit, matplotlib e numpy**.

Estrutura do Projecto

O nosso projecto segue a seguinte estrutura principal (onde enumeramos posteriormente por questões de ordem): **project/{data, scripts, results, plots, report}**

1 QUESTÃO A

Pretende-se Gerar arquivos de texto aleatórios com os seguintes tamanhos(em Bytes): 8, 64, 512, 4096, 32768, 262144, 2097152. As condições com as quais esta tarefa deve ser efetuada foram descritas no enunciado do Trabalho prático.

Nesta etapa, nosso objectivo é criar uma base limpa, controlada e reproduzível para todo o benchmarking.
Escolhemos o uso de "os.random()"em nosso programa para a geração aleatória dos dados pois, esta abordagem garante que os dados não apresentem padrões previsíveis, tornando-os mais representativos de situações reais onde a informação processada é arbitrária e de alta entropia. Importa observar que a utilização de dados aleatórios evita vieses que poderiam surgir caso fossem utilizados ficheiros com padrões repetitivos ou compressíveis, os quais poderiam influenciar o comportamento de certos algoritmos.

Conforme orientado pelo enunciado do trabalho, separamos (em ficheiro de código) a "geração de dados" da "execução dos benchmarks" para assim assegurar que os tempos medidos correspondem exclusivamente às operações criptográficas (posterores).
Assim, para o primeiro script do nosso programa em Python cujos objectivos referimos acima (gerar ficheiros binários com diferentes tamanhos, contendo dados aleatórios seguros), nomeamos-lo de "`0_generate_files` ".

1.1 Código

Parte principal do código `0_generate_files`:

```python
import os # Modulo para interagir com o sistema operacional, como criar diretorios e arquivos

# Tamanhos em bytes
file_sizes = [8, 64, 512, 4096, 32768, 262144, 2097152]

```

```python
output_dir = "../1_data" # Diretorio onde os ficheiros serao salvos

os.makedirs(output_dir, exist_ok=True) # Cria o diretorio se ele nao existir

for size in file_sizes: # Itera sobre os tamanhos definidos para criar os ficheiros
    filename = f"{output_dir}/file_{size}B.bin"

    # Dados aleatorios seguros
    data = os.urandom(size)

     with open(filename, "wb") as f: # Abre o arquivo para escrita em modo binario
        f.write(data) # Escreve os dados aleatorios no arquivo
```

Listing 1: Programa em Python responsável por Gerar ficheiros binários com dados aleatórios

Tendo o programa pronto, executamos e obtemos os seguintes resultados:

1.2 Resultado

```
1º Ficheiro criado: ../1_data/file_8B.bin (8 bytes)
2º Ficheiro criado: ../1_data/file_64B.bin (64 bytes)
3º Ficheiro criado: ../1_data/file_512B.bin (512 bytes)
4º Ficheiro criado: ../1_data/file_4096B.bin (4096 bytes)
5º Ficheiro criado: ../1_data/file_32768B.bin (32768 bytes)
6º Ficheiro criado: ../1_data/file_262144B.bin (262144 bytes)
7º Ficheiro criado: ../1_data/file_2097152B.bin (2097152 bytes)

--- Todos os ficheiros foram gerados com sucesso. ---
```

Figura 1: `0_generate_files`

Com isto, temos a questão resolvida.

2 QUESTÃO B

Nesta questão, implementamos o algoritmo de criptografia simétrica AES (Advanced Encryption Standard), nosso objectivo é medir o seu desempenho(tempo médio e o desvio padrão) na cifragem e decifragem de ficheiros de diferentes dimensões (os quais geramos na questão anterior).

Assim, para o segundo script do nosso programa em Python cujos objectivos referimos acima (medir o desempenho do algoritmo de criptografia simétrica AES na cifragem e decifragem), nomeamos-lo de "`1_aes_test`".

2.1 Código

Parte principal do código `1_aes_test`:

```python
import os
import time
import numpy as np
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

# Configuracao
DATA_DIR = "../1_data"
RESULTS = []

# Chave AES-256 (32 bytes)
KEY = os.urandom(32)  # Em producao, deve ser armazenada com seguran a

def aes_encrypt(data):
    # Gerar nonce aleatorio (unico por operacao)
    nonce = os.urandom(16)
    cipher = Cipher(algorithms.AES(KEY), modes.CTR(nonce))
    encryptor = cipher.encryptor()
    return nonce + encryptor.update(data) + encryptor.finalize()

def aes_decrypt(ciphertext):
    # Extrair nonce do inicio
    nonce = ciphertext[:16]
    ...

def benchmark(file_path, runs=30):
    with open(file_path, "rb") as f:
        data = f.read()
```

```python

    enc_times = []
    dec_times = []

    for _ in range(runs):
        # Tempo de encriptacao
        start = time.perf_counter()
        ciphertext = aes_encrypt(data)
        end = time.perf_counter()
        enc_times.append((end - start) * 1e6)  # microssegundos

        # Tempo de desencriptacao
        start = time.perf_counter()
        ...

    return {
        "file": os.path.basename(file_path),
        "size": len(data),
        "enc_avg": np.mean(enc_times),
        ...
    }

def main():
    # Ordenar ficheiros por tamanho
    files = sorted(
        os.listdir(DATA_DIR),
        key=lambda x: int(x.split('_')[1][:-5])
    )
    ...

    for file in files:
        path = os.path.join(DATA_DIR, file)
        result = benchmark(path)
        RESULTS.append(result)

        ...

    # Guardar resultados
    np.save("../3_results/1_aes_results.npy", RESULTS)
    ...
```

Listing 2: Benchmark de criptografia AES-256 em modo CTR

2.2 Implementação, Descrição e Avaliação do AES

A implementação foi realizada com recurso à biblioteca cryptography, que disponibiliza primitivas criptográficas seguras e otimizadas. Foi utilizado o algoritmo AES com chave de 256 bits (AES-256), garantindo um elevado nível de segurança.

O modo de operação adotado foi o CTR (Counter Mode), no qual, para cada operação de cifragem, é gerado um valor aleatório denominado nonce. Este elemento é fundamental para assegurar a segurança do processo, evitando a reutilização do fluxo criptográfico.

O processo de encriptação inicia-se com a inicialização do algoritmo AES utilizando a chave secreta e o nonce gerado. Em seguida, é aplicado o fluxo de cifragem aos dados de entrada, resultando na produção do texto cifrado. O nonce é concatenado ao texto cifrado, permitindo que esteja disponível para o processo de desencriptação.

A desencriptação segue o processo inverso, utilizando a mesma chave e o nonce associado, possibilitando a recuperação correta dos dados originais.

Para avaliar o desempenho do algoritmo, foram realizados testes sobre os ficheiros gerados na fase anterior. Cada ficheiro foi processado por ordem crescente de tamanho, sendo executadas múltiplas iterações (por defeito, 30) para garantir maior estabilidade nas medições.

As medições de tempo foram realizadas com recurso a um temporizador de alta precisão (time.perf_counter), sendo registados os tempos de encriptação e desencriptação em microssegundos. Com base nesses valores, foram calculadas métricas estatísticas, nomeadamente a média e o desvio padrão.

Importa salientar que os tempos associados à leitura e escrita de ficheiros foram excluídos das medições, assegurando que os resultados refletem exclusivamente o desempenho das operações criptográficas. Todos os resultados obtidos foram armazenados em ficheiros no formato .npy, permitindo posterior análise.

Tendo o programa pronto, executamos e obtemos os seguintes resultados:

2.3 Resultados do Benchmark AES-256

- **Primeira Execução:**

A Tabela 1 apresenta os tempos médios de encriptação e desencriptação, bem como os respetivos desvios padrão, para ficheiros de diferentes tamanhos.

Tabela 1: Resultados do benchmark de criptografia AES-256

Arquivo	**Encrypt (µs)**	**± Encrypt**	**Decrypt (µs)**	**± Decrypt**
file_8B.bin	2162.11	11494.91	30.92	25.69
file_64B.bin	37.04	19.33	32.49	2.75
file_512B.bin	37.91	25.48	32.54	12.07
file_4096B.bin	59.84	27.55	52.78	16.01
file_32768B.bin	84.63	46.52	75.17	14.49
file_262144B.bin	983.75	510.16	448.26	344.76
file_2097152B.bin	3565.73	236.25	3472.17	272.32

- **Segunda Execução:**

Tabela 2: Resultados do benchmark de criptografia AES-256 (Execução 2)

Arquivo	**Encrypt (µs)**	**± Encrypt**	**Decrypt (µs)**	**± Decrypt**
file_8B.bin	1444.60	7353.20	84.43	72.81
file_64B.bin	80.99	25.77	71.88	13.50
file_512B.bin	58.88	27.39	52.53	10.58
file_4096B.bin	65.93	64.44	57.39	42.08
file_32768B.bin	68.17	47.81	56.77	20.45
file_262144B.bin	273.11	226.08	232.94	109.65
file_2097152B.bin	3812.18	566.90	3608.42	587.28

- **Sexta Execução:**

Tabela 3: Resultados do benchmark de criptografia AES-256 (Execução 6)

Arquivo	**Encrypt (µs)**	**± Encrypt**	**Decrypt (µs)**	**± Decrypt**
file_8B.bin	79.81	308.15	22.68	5.22
file_64B.bin	37.39	24.00	31.80	8.78
file_512B.bin	38.14	41.92	27.93	10.87
file_4096B.bin	35.06	24.14	28.74	11.81
file_32768B.bin	44.52	29.76	39.47	6.07
file_262144B.bin	527.01	453.52	397.53	535.30
file_2097152B.bin	3930.35	990.53	4029.50	1507.37

Com isto, temos a questão resolvida.

3 QUESTÃO C

Esta questão consiste em implementar um mecanismo de criptografia baseado no algoritmo RSA, complementado com a função de hash SHA-256, conforme pedido no enunciado.

3.1 Código

Este script em Python implementa e avalia o desempenho de um esquema de criptografia baseado em RSA com pre-enchimento OAEP(Optimal Asymmetric Encryption Padding), adaptado para suportar ficheiros de tamanho arbitrário através de uma abordagem híbrida baseada em blocos e função de hash.

```python
import os
import time
import numpy as np
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

DATA_DIR = "../1_data"
RESULTS = []

```

```python
# Gerar chave RSA (uma vez)
PRIVATE_KEY = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
PUBLIC_KEY = PRIVATE_KEY.public_key()

BLOCK_SIZE = 32  # SHA256 output = 32 bytes

def sha256(data):
    digest = hashes.Hash(hashes.SHA256())
    digest.update(data)
    return digest.finalize()

def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

# Encrypt
def rsa_encrypt(data):
    r = os.urandom(32)

    # RSA(r)
    enc_r = PUBLIC_KEY.encrypt(
        r,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    ciphertext_blocks = []

    ...

    return enc_r, b''.join(ciphertext_blocks)

# Decrypt
def rsa_decrypt(enc_r, ciphertext):
    r = PRIVATE_KEY.decrypt(
        enc_r,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    ...


def benchmark(file_path, runs=3):
    with open(file_path, "rb") as f:
        data = f.read()

    enc_times = []
    dec_times = []

    for _ in range(runs):
        # Encrypt
        start = time.perf_counter()
        enc_r, ciphertext = rsa_encrypt(data)
        end = time.perf_counter()
        enc_times.append((end - start) * 1e6)

        # Decrypt
        start = time.perf_counter()
        _ = rsa_decrypt(enc_r, ciphertext)
        end = time.perf_counter()
        dec_times.append((end - start) * 1e6)

    return {
        "file": os.path.basename(file_path),
        "size": len(data),
        "enc_avg": np.mean(enc_times),
        "enc_std": np.std(enc_times),
        "dec_avg": np.mean(dec_times),
        "dec_std": np.std(dec_times),
    }
```

```
        ...

if __name__ == "__main__":
    main()
```

Listing 3: Benchmark de criptografia RSA com esquema híbrido baseado em SHA-256

3.2 Funcionamento e Descrição do Algoritmo

O sistema implementado utiliza um esquema de encriptação híbrido, combinando criptografia assimétrica e simétrica para garantir segurança e eficiência no processamento de dados.

Inicialmente, é gerado um valor aleatório "r", que é posteriormente cifrado utilizando o algoritmo RSA com chave de 2048 bits e preenchimento OAEP baseado em SHA-256. O uso do RSA é restrito à proteção deste valor aleatório, reduzindo significativamente o custo computacional associado ao tratamento de grandes volumes de dados.

A mensagem a ser cifrada é dividida em blocos de tamanho fixo (32 bytes). Para cada bloco, é gerado um fluxo pseudoaleatório através da aplicação da função de hash SHA-256 ao valor "r" e ao índice do bloco. Este fluxo é então combinado com o bloco correspondente através da operação XOR, resultando no bloco cifrado.

O resultado final do processo de encriptação é composto por dois elementos principais:

- **O valor "r" cifrado com RSA;**
- **Os blocos de dados cifrados com base no fluxo pseudoaleatório gerado.**

O processo de desencriptação segue exatamente os mesmos passos em ordem inversa, utilizando a chave privada para recuperar o valor "r" e, posteriormente, reconstruir o fluxo pseudoaleatório necessário para aplicar a operação XOR e obter os dados originais.

Para avaliação de desempenho, são realizadas múltiplas execuções do algoritmo, permitindo o cálculo de métricas estatísticas como tempo médio e desvio padrão. Os resultados obtidos são armazenados em ficheiros no formato ".npy".

3.3 Resultados do benchmark de criptografia RSA

- **Primeira Execução:**

Tabela 4: Resultados do benchmark de criptografia RSA (Execução 1)

Arquivo	**Encrypt (µs)**	**± Encrypt**	**Decrypt (µs)**	**± Decrypt**
file_8B.bin	13841.53	19294.90	5642.90	5550.06
file_64B.bin	254.73	52.45	1594.33	124.78
file_512B.bin	597.30	276.70	1604.43	273.65
file_4096B.bin	4997.93	1988.83	8277.10	1343.91
file_32768B.bin	28205.60	10533.70	23540.50	10858.71
file_262144B.bin	205722.97	22733.57	181546.03	40204.48
file_2097152B.bin	1167678.23	154487.75	1186931.90	170969.51

- **Segunda Execução:**

Tabela 5: Resultados do benchmark de criptografia RSA (Execução 2)

Arquivo	**Encrypt (µs)**	**± Encrypt**	**Decrypt (µs)**	**± Decrypt**
file_8B.bin	584.03	696.94	1371.40	591.06
file_64B.bin	129.17	41.18	993.97	38.30
file_512B.bin	545.37	395.64	1528.93	475.14
file_4096B.bin	3297.47	1391.72	3383.67	594.94
file_32768B.bin	22943.13	6268.33	22569.13	1895.99
file_262144B.bin	142246.80	4370.44	140084.67	10612.31
file_2097152B.bin	1309842.10	163349.39	1253948.80	100900.45

- **Sexta Execução:**

Tabela 6: Resultados do benchmark de criptografia RSA (Execução 6)

Arquivo	Encrypt (µs)	± Encrypt	Decrypt (µs)	± Decrypt
file_8B.bin	674.80	758.81	1240.23	464.91
file_64B.bin	307.43	45.73	3311.13	2705.52
file_512B.bin	274.13	47.79	1327.10	194.88
file_4096B.bin	2944.63	1845.36	5433.10	2623.13
file_32768B.bin	21667.10	7043.64	20832.20	7759.95
file_262144B.bin	164454.77	21337.73	206242.60	90628.78
file_2097152B.bin	1155460.00	46244.78	1168157.87	14527.77

4 QUESTÃO D

Nesta fase foi analisado o desempenho da função de hash criptográfica SHA-256 (Secure Hash Algorithm 256 bits), com o objetivo de medir o tempo necessário para gerar resumos (digests) de ficheiros com diferentes dimensões.

4.1 Código

Este script em Python avalia o desempenho da função de hash SHA-256 aplicada a ficheiros de diferentes tamanhos. Com o mesmo objectivo dos scripts anteriores.

```python
import os
import time
import numpy as np
from cryptography.hazmat.primitives import hashes

DATA_DIR = "../1_data"
RESULTS = []

def sha256(data):
    digest = hashes.Hash(hashes.SHA256())
    digest.update(data)
    return digest.finalize()

def benchmark(file_path, runs=30):
    with open(file_path, "rb") as f:
        data = f.read()

    times = []

    for _ in range(runs):
        start = time.perf_counter()
        _ = sha256(data)
        end = time.perf_counter()

        times.append((end - start) * 1e6)  # microssegundos

    return {
        ...
    }

def main():
    files = sorted(...
    )

   ... ...
```

Listing 4: Benchmark da função de hash SHA-256

4.2 Descrição, Implementação e Avaliação do SHA-256

O SHA-256 é uma função de hash criptográfica amplamente utilizada, caracterizada por produzir um output de tamanho fixo de 256 bits. Trata-se de uma função determinística, ou seja, a mesma entrada origina sempre o mesmo resultado. Adicionalmente, apresenta resistência a colisões, tornando computacionalmente inviável encontrar duas entradas distintas com o mesmo hash, e possui natureza unidirecional, não sendo possível reverter o processo para obter os dados originais a partir do digest.

A implementação foi realizada em Python com recurso à biblioteca cryptography, assegurando o uso de primitivas seguras e eficientes. Para cada ficheiro, o cálculo do hash segue um processo simples: inicia-se o contexto de hash, processam-se os dados de entrada e, por fim, gera-se o digest final. Este procedimento é aplicado diretamente sobre os dados completos, não sendo necessária a divisão manual em blocos, uma vez que essa gestão é tratada internamente pela biblioteca.

Para avaliar o desempenho do SHA-256, foram realizados testes sobre os ficheiros em análise, processados por ordem crescente de tamanho. O conteúdo de cada ficheiro é carregado em memória e o cálculo do hash é repetido múltiplas vezes (por defeito, 30 execuções), permitindo obter medições mais consistentes.

Os tempos de execução são registados em microssegundos, utilizando um temporizador de alta precisão. Com base nesses valores, são calculadas métricas estatísticas, nomeadamente a média e o desvio padrão. À semelhança das restantes componentes do sistema, os resultados são armazenados em ficheiros no formato ".npy", possibilitando análise posterior.

4.3 Resultados do benchmark da função de hash SHA-256

Primeira Execução

Arquivo	Hash (µs)	± Hash
file_8B.bin	1978.06	10580.43
file_64B.bin	8.72	10.66
file_512B.bin	11.20	12.66
file_4096B.bin	34.35	19.11
file_32768B.bin	133.14	30.47
file_262144B.bin	1445.89	550.09
file_2097152B.bin	9781.26	1736.06

Segunda Execução

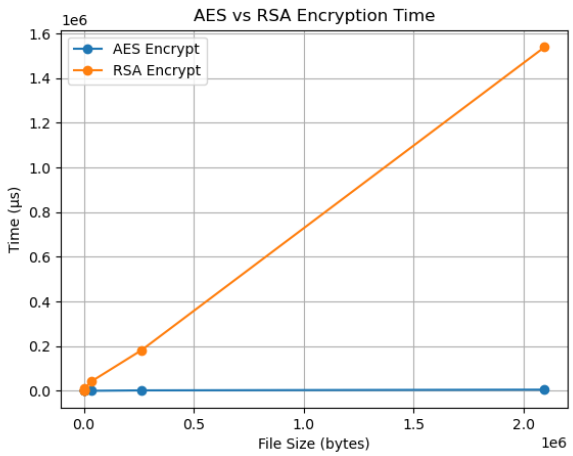
Arquivo	Hash (µs)	± Hash
file_8B.bin	105.58	505.59
file_64B.bin	13.08	9.63
file_512B.bin	15.28	8.11
file_4096B.bin	34.73	7.42
file_32768B.bin	247.68	77.10
file_262144B.bin	2366.19	1121.94
file_2097152B.bin	19879.18	13287.64

Sexta Execução

Arquivo	Hash (µs)	± Hash
file_8B.bin	62.09	289.76
file_64B.bin	5.73	7.25
file_512B.bin	14.30	13.63
file_4096B.bin	32.50	12.62
file_32768B.bin	192.09	46.82
file_262144B.bin	946.16	213.57
file_2097152B.bin	6764.88	1175.48

Tabela 7: Comparação entre duas execuções do benchmark SHA-256

5 GRÁFICOS COMPARATIVOS DE DESEMPENHO (Geral)



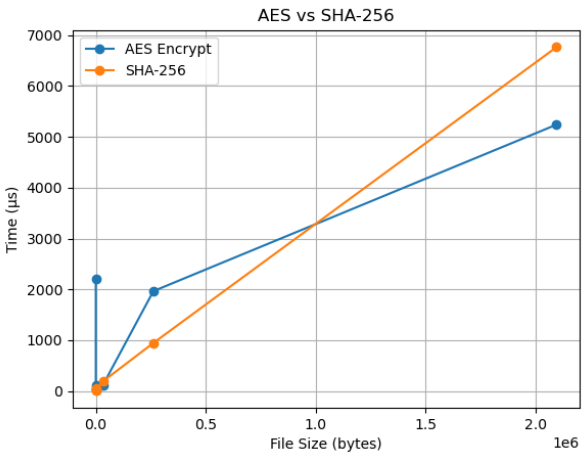


Figura 2: Comparação dos algoritmos

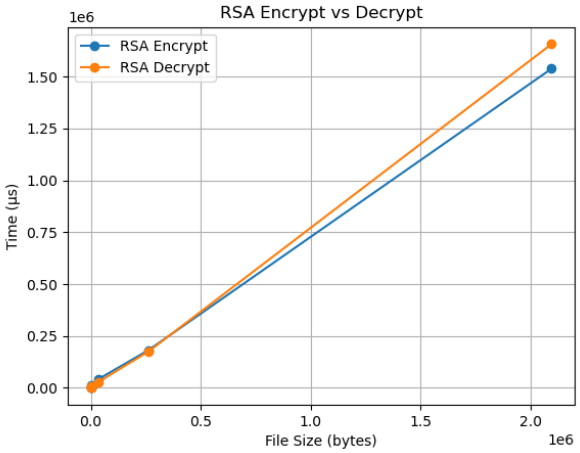


Figura 3: Comparação RSA: Encrypt vs Decrypt

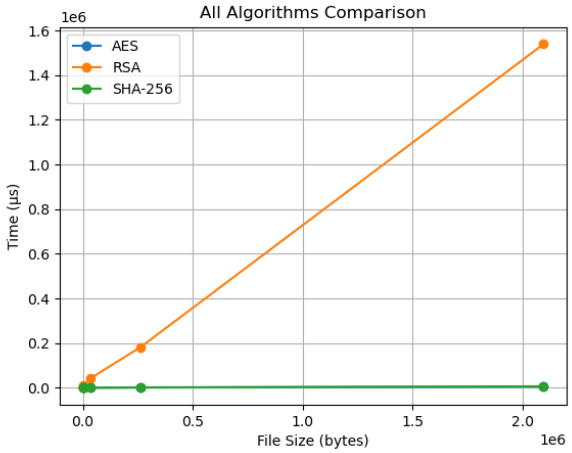


Figura 4: Comparação final de todos os algoritmos

ANÁLISE GERAL E CONCLUSÃO

Algoritmo AES

Durante os testes, notamos que: O AES demonstrou comportamento linear em relação ao tamanho dos ficheiros, com tempos de encriptação e decriptação muito semelhantes(excepto para o arquivo de minimo tamanho), como esperado para o modo CTR. Este resultado confirma a eficiência do AES para processamento de grandes volumes de dados.

Algoritmo RSA

Durante os testes, notamos que:

- ficheiros pequenos, quase iguais (8B, 64B, 512B) consumiam tempos muito parecidos;
- ficheiros grandes(4096B, 2MB) explodiam, retornando crescimento forte;
- Crescimento não totalmente linear.

É notório um desempenho mais reduzido do RSA em relação ao AES, sendo um comportamento esperado por se tratar de um algoritmo computacionalmente intensivo, sendo necessário reduzir o número de execuções para 10, evitando tempos de processamento excessivos. Porém, mesmo com esta redução, notamos que o sistema era demasiado demorado(parecia travado), ao que reduzimos para 3 e excluimos inicialmente o ficheiro de maior tamanho (2MB) para testes iniciais. **Confirmamos portanto que RSA não foi feito para cifrar dados grandes.**

Em geral, nota-se também que, em RSA, a decriptação costuma ser mais lenta. Os valores altos do desvio padrão mostram a estabilidade do sistema e o ruído de CPU/cache.

O RSA é MUITO mais lento que AES, com uma diferença de $\sim$ 100x ou mais.

Algoritmo SHA-256

Durante os testes, notamos que:

- Apresenta Crescimento linear: Quanto maior o ficheiro $\rightarrow$ maior o tempo(hash percorre todos os dados);
- É muito mais rápido que RSA;
- SHA-256 não cifra, apenas gera um digest(apenas processamento linear);
- Apresenta pequena variação (std baixo).

Para um ficheiro de 2 MB, os tempos de execução observados evidenciam diferenças significativas entre os algoritmos analisados. O SHA-256 apresentou o melhor desempenho, com um tempo $\sim 1000(\mu s)$. Em seguida, o AES registou um tempo médio de $\sim 4000(\mu s)$, mantendo ainda uma execução bastante eficiente. Por fim, o RSA demonstrou ser substancialmente mais lento, com um tempo de $\sim 150000(\mu s)$, refletindo o seu maior custo computacional (com estas diferenças brutais).

Respondendo as Questões do enunciado:

- **Execuções repetidas:**
  - Observou-se alguma variação nos tempos devido a fatores como cache e carga do sistema, sendo por isso essencial a realização de múltiplas execuções.

- **Ficheiros diferentes vs iguais:**
  - Os tempos mantêm-se consistentes para ficheiros do mesmo tamanho, indicando que o conteúdo dos dados não influencia significativamente o desempenho.

Tabela 8: Comparação entre algoritmos criptográficos

Algoritmo	**Objetivo**	**Desempenho / Características**
AES	Confidencialidade	Muito rápido; mais eficiente que RSA, especialmente para grandes volumes de dados
RSA	Troca segura	Mais lento devido a operações complexas; desencriptação mais custosa que encriptação
SHA-256	Integridade	Mais rápido que AES; não cifra dados, apenas gera hash (sem confidencialidade)

Considerações

Apesar de teoricamente funções de hash como SHA-256 serem mais simples do que algoritmos de cifragem, os resultados mostram que o AES apresenta melhor desempenho em ficheiros grandes. Isto deve-se provavelmente ao uso de aceleração por hardware (AES-NI), que otimiza significativamente operações de cifragem simétrica em CPUs modernas.

Este trabalho permitiu analisar experimentalmente o desempenho de diferentes mecanismos criptográficos, evidenciando as vantagens dos algoritmos simétricos para processamento de dados em larga escala e demonstrando que factores como otimizações de hardware podem influenciar significativamente os resultados práticos.

Informações Adicionais

- **Carcterísticas da Máquina/Computador de realização dos testes:** A Tabela asseguir apresenta as características do ambiente de execução utilizado nos testes.

Tabela 9: Especificações do sistema utilizado

Componente	**Especificação**
Modelo	Dell Latitude 7280
Sistema Operacional	Windows 10 Pro (22H2)
Processador	Intel Core i5-6200U @ 2.30GHz
Memória RAM	8 GB (2400 MHz)
Placa Gráfica	Intel HD Graphics 520
Armazenamento	238 GB (220 GB usados)
OS Build	19045.7058

- **Referências usadas:**
  - https://moodle2526.up.pt/course/view.php?id=6089 -Apontamentos das aulas Teóricas e Práticas
  - https://www.youtube.com/watch?v=pOx2TYwR590&t=2s - Implementing hybrid RSA-AES encryption in Python
  - https://www.youtube.com/watch?v=tyhl7EYmJoM - Implementing AES in Python
  - https://www.youtube.com/watch?v=XmljxJJJl1w - How to Encrypt with RSA using Python
  - https://www.youtube.com/watch?v=U3jEmc-L58Y - Implementing SHA-256 in Python
  - https://www.google.com/ - Consultas e conceituais
  - https://chatgpt.com/ - Consultas e correções de códigos
- **Link para o nosso repositório GitHub:**
  - https://github.com/PedrobGraca/TP1_SP - Conforme orientado, aqui se encontram os arquivos completos do projecto.